

01

ELIMINAR UN ELEMENTO "X"

MATRICES • JAVA • ALGORITMOS • SEMANA 02

■ CONCEPTO CENTRAL

Localizar todas las apariciones de un valor X y sustituirlas por 0.

■ ¿CÓMO FUNCIONA?

El algoritmo utiliza dos bucles anidados. El primer bucle recorre las filas y el segundo las columnas. En cada posición se compara el contenido con X. Si son iguales, la celda se reemplaza por 0.

■ PASO A PASO PARA EXPONER

SECUENCIA DEL ALGORITMO

1) Iniciar el recorrido. 2) Leer matriz[i][j]. 3) Comparar con X. 4) Si coincide, asignar 0. 5) Continuar hasta revisar toda la matriz.

■ CÓDIGO / LÓGICA

```
for (i = 0; i < fila; i++) {  
    for (j = 0; j < columna; j++) {  
        if (matriz[i][j] == x) {  
            matriz[i][j] = 0;  
        }  
    }  
}
```

■ ANTES → DESPUÉS

ESTADO INICIAL

X	8	3
5	X	1
4	7	X



RESULTADO

0	8	3
5	0	1
4	7	0

■ EJEMPLO PARA DECIR EN CLASE

En el ejemplo existen tres X. Por eso aparecen tres ceros en las mismas posiciones. No desaparece una fila ni una columna: se cambia el contenido de las celdas encontradas.

★ IDEA CLAVE

El doble recorrido garantiza que se revise cada casilla de la estructura bidimensional.

★ ESTRUCTURA

Filas + columnas + condición/recorrido

★ RESULTADO

Se transforma o analiza la matriz

■ CONCEPTO CENTRAL

Encontrar un valor objetivo y reemplazarlo por un nuevo valor.

■ ¿CÓMO FUNCIONA?

El método recorre la matriz y compara cada elemento con el valor buscado. Cuando encuentra una coincidencia, realiza una asignación con el nuevo valor.

■ PASO A PASO PARA EXPONER

SECUENCIA DEL ALGORITMO

1) Definir X. 2) Definir nuevoValor. 3) Recorrer filas y columnas. 4) Comparar. 5) Reemplazar las coincidencias.

■ CÓDIGO / LÓGICA

```
if (matriz[i][j] == x) {  
    matriz[i][j] = nuevoValor;  
}
```

■ ANTES → DESPUÉS

ESTADO INICIAL

7	2
4	7



RESULTADO

99	2
4	99

■ EJEMPLO PARA DECIR EN CLASE

Si buscamos 7 y el nuevo valor es 99, ambas apariciones de 7 se transforman en 99. Esto demuestra que la modificación puede afectar varias posiciones.

★ IDEA CLAVE

La diferencia principal con eliminar es el valor de reemplazo: aquí se utiliza nuevoValor.

★ ESTRUCTURA

Filas + columnas + condición/recorrido

★ RESULTADO

Se transforma o analiza la matriz

■ CONCEPTO CENTRAL

Obtener una única suma con todos los elementos almacenados.

■ ¿CÓMO FUNCIONA?

La variable suma funciona como acumulador. Comienza en cero y recibe cada elemento de la matriz durante el recorrido. Al finalizar contiene el total.

■ PASO A PASO PARA EXPONER

SECUENCIA DEL ALGORITMO

1) suma = 0. 2) Recorrer filas. 3) Recorrer columnas. 4) Agregar cada elemento. 5) Mostrar suma.

■ CÓDIGO / LÓGICA

```
int suma = 0;

for (i = 0; i < fila; i++) {
  for (j = 0; j < columna; j++) {
    suma += matriz[i][j];
  }
}
```

■ ANTES → DESPUÉS

ESTADO INICIAL

5	2	1
4	3	6
8	0	2



RESULTADO

SUMA	31
------	----

■ EJEMPLO PARA DECIR EN CLASE

$5 + 2 + 1 + 4 + 3 + 6 + 8 + 0 + 2 = 31$. Cada elemento se suma una sola vez.

★ IDEA CLAVE

El acumulador es la pieza central del algoritmo: conserva el resultado parcial mientras avanza el recorrido.

★ ESTRUCTURA

Filas + columnas + condición/recorrido

★ RESULTADO

Se transforma o analiza la matriz

■ CONCEPTO CENTRAL

Calcular estadísticas globales de la matriz: mínimo, máximo y promedio.

■ ¿CÓMO FUNCIONA?

El menor y el mayor se obtienen mediante comparaciones. El promedio utiliza la suma total y la cantidad de elementos: filas por columnas.

■ PASO A PASO PARA EXPONER

SECUENCIA DEL ALGORITMO

1) Inicializar comparadores. 2) Recorrer. 3) Actualizar menor/mayor. 4) Acumular suma. 5) Dividir suma entre total.

■ CÓDIGO / LÓGICA

```
int total = fila * columna;  
double promedio = (double) suma / total;
```

■ ANTES → DESPUÉS

ESTADO INICIAL

5	2	1
4	3	6
8	0	2



RESULTADO

MENOR	0
MAYOR	8
PROM	3.44

■ EJEMPLO PARA DECIR EN CLASE

En la matriz del ejemplo el menor es 0 y el mayor es 8. Con la suma 31 y nueve elementos, el promedio aproximado es 3.44.

★ IDEA CLAVE

Este método combina tres operaciones frecuentes: comparación, acumulación y división.

★ ESTRUCTURA

Filas + columnas + condición/recorrido

★ RESULTADO

Se transforma o analiza la matriz

■ CONCEPTO CENTRAL

Sumar las posiciones que forman visualmente una letra L.

■ ¿CÓMO FUNCIONA?

El recorrido toma la primera columna y luego la última fila. La segunda parte comienza en $j = 1$ para evitar volver a sumar la esquina inferior izquierda.

■ PASO A PASO PARA EXPONER

SECUENCIA DEL ALGORITMO

1) Bajar por la primera columna. 2) Llegar a la esquina. 3) Avanzar por la última fila. 4) Acumular los valores.

■ CÓDIGO / LÓGICA

```
for (i = 0; i < fila; i++)
    suma += matriz[i][0];

for (j = 1; j < columna; j++)
    suma += matriz[fila-1][j];
```

■ ANTES → DESPUÉS

ESTADO INICIAL

5	0	0
3	0	0
2	8	4



RESULTADO

SUMA	22
------	----

■ EJEMPLO PARA DECIR EN CLASE

Las celdas utilizadas forman dos segmentos perpendiculares. Este patrón permite reconocer visualmente una L dentro de la matriz.

★ IDEA CLAVE

La condición de inicio del segundo recorrido evita una duplicación.

★ ESTRUCTURA

Filas + columnas + condición/recorrido

★ RESULTADO

Se transforma o analiza la matriz

■ CONCEPTO CENTRAL

Realizar un recorrido que represente una letra J y sumar sus posiciones.

■ ¿CÓMO FUNCIONA?

El método se basa en reconocer un patrón de recorrido: comienza en la parte superior, continúa por el extremo derecho y completa la parte inferior según el esquema.

■ PASO A PASO PARA EXPONER

SECUENCIA DEL ALGORITMO

1) Identificar el borde superior. 2) Recorrer el extremo. 3) Completar la base. 4) Evitar contar una posición compartida dos veces.

■ CÓDIGO / LÓGICA

```
// fila superior completa  
// última columna  
// base inferior  
// controlar posiciones repetidas
```

■ ANTES → DESPUÉS

ESTADO INICIAL

1	2	3
4	0	0
6	7	0



RESULTADO

SUMA	23
------	----

■ EJEMPLO PARA DECIR EN CLASE

En una exposición conviene marcar las celdas participantes con flechas. El objetivo es comprender que una matriz también puede recorrerse siguiendo figuras.

★ IDEA CLAVE

Lo importante es la lógica del recorrido, no recorrer todas las celdas indiscriminadamente.

★ ESTRUCTURA

Filas + columnas + condición/recorrido

★ RESULTADO

Se transforma o analiza la matriz

■ CONCEPTO CENTRAL

Sumar los dos laterales y la base inferior formando una U.

■ ¿CÓMO FUNCIONA?

Se recorren el lateral izquierdo y el lateral derecho. Después se completa la base inferior. Las esquinas pertenecen a dos segmentos, por lo que deben controlarse.

■ PASO A PASO PARA EXPONER

SECUENCIA DEL ALGORITMO

1) Recorrer columna izquierda. 2) Recorrer columna derecha. 3) Recorrer la base. 4) Controlar las esquinas.

■ CÓDIGO / LÓGICA

```
// lateral izquierdo  
// lateral derecho  
// base inferior  
// evitar repetir esquinas
```

■ ANTES → DESPUÉS

ESTADO INICIAL

9	0	5
4	0	0
1	7	5
3	0	0
2	0	0



RESULTADO

SUMA	31
------	----

■ EJEMPLO PARA DECIR EN CLASE

El recorrido representa tres lados de un rectángulo: izquierda, derecha y abajo. Los elementos centrales no participan.

★ IDEA CLAVE

Los límites de filas y columnas permiten construir patrones geométricos.

★ ESTRUCTURA

Filas + columnas + condición/recorrido

★ RESULTADO

Se transforma o analiza la matriz

■ CONCEPTO CENTRAL

Encontrar un mínimo diferente para cada fila de la matriz.

■ ¿CÓMO FUNCIONA?

Cada fila tiene su propio proceso de comparación. Al comenzar una nueva fila, menor se reinicia tomando la primera celda como referencia.

■ PASO A PASO PARA EXPONER

SECUENCIA DEL ALGORITMO

1) Seleccionar fila. 2) Tomar primera celda. 3) Comparar restantes. 4) Actualizar menor. 5) Guardar o mostrar resultado.

■ CÓDIGO / LÓGICA

```
for (i = 0; i < fila; i++) {  
    menor = matriz[i][0];  
  
    for (j = 1; j < columna; j++) {  
        if (matriz[i][j] < menor)  
            menor = matriz[i][j];  
    }  
}
```

■ ANTES → DESPUÉS

ESTADO INICIAL

8	3	5
12	19	7
4	1	9



RESULTADO

FILA 0	3
FILA 1	7
FILA 2	1

■ EJEMPLO PARA DECIR EN CLASE

Fila 0 → 3. Fila 1 → 7. Fila 2 → 1. No se busca un único mínimo global, sino uno independiente por cada fila.

★ IDEA CLAVE

La variable menor debe reiniciarse para que cada fila tenga su propio resultado.

★ ESTRUCTURA

Filas + columnas + condición/recorrido

★ RESULTADO

Se transforma o analiza la matriz

■ CONCEPTO CENTRAL

Contar cuántos elementos de la matriz son pares.

■ ¿CÓMO FUNCIONA?

El operador % devuelve el residuo de una división. Si el residuo de dividir entre 2 es cero, el número es par y el contador aumenta.

■ PASO A PASO PARA EXPONER

SECUENCIA DEL ALGORITMO

1) Iniciar contador en 0. 2) Recorrer cada celda. 3) Aplicar % 2. 4) Si es 0, incrementar. 5) Mostrar contador.

■ CÓDIGO / LÓGICA

```
int contador = 0;

if (matriz[i][j] % 2 == 0) {
    contador++;
}
```

■ ANTES → DESPUÉS

ESTADO INICIAL

4	7	2
9	6	11
10	8	3



RESULTADO

PARS	2, 6, 10, 8
TOTAL	5

■ EJEMPLO PARA DECIR EN CLASE

Los pares del ejemplo son 4, 2, 6, 10 y 8. Por tanto, el total de pares es 5.

★ IDEA CLAVE

Es un ejemplo de filtrado: de todos los datos, solo se cuentan los que cumplen una condición.

★ ESTRUCTURA

Filas + columnas + condición/recorrido

★ RESULTADO

Se transforma o analiza la matriz

■ CONCEPTO CENTRAL

Desplazar los elementos de cada fila una posición a la derecha conservando el último valor.

■ ¿CÓMO FUNCIONA?

Primero se guarda el último elemento en una variable temporal. Después se desplazan los elementos desde la derecha hacia la izquierda y finalmente el valor guardado se coloca en la primera posición.

■ PASO A PASO PARA EXPONER

SECUENCIA DEL ALGORITMO

1) Guardar último. 2) Desplazar de derecha a izquierda. 3) Colocar temporal en $j=0$. 4) Repetir para cada fila.

■ CÓDIGO / LÓGICA

```
temporal = matriz[i][columna - 1];  
  
for (j = columna - 1; j > 0; j--) {  
    matriz[i][j] = matriz[i][j - 1];  
}  
  
matriz[i][0] = temporal;
```

■ ANTES → DESPUÉS

ESTADO INICIAL



RESULTADO



■ EJEMPLO PARA DECIR EN CLASE

Antes: A B C D. Después: D A B C. El último elemento pasa al inicio, por eso la rotación es circular.

★ IDEA CLAVE

El orden del desplazamiento es importante: comenzar por el final evita sobrescribir datos que todavía deben moverse.

★ ESTRUCTURA

Filas + columnas + condición/recorrido

★ RESULTADO

Se transforma o analiza la matriz